

软件工程概论

张璇 云南大学

高水平文献阅读

- *Systematic Mapping Study* is a form of secondary study aiming to get a comprehensive overview on a certain research topic, to identify research gaps, and to collect evidence in order to direct future research (Kitchenham and Charters, 2007, Engström and Runeson, 2011);
- *Systematic Literature Reviews* (SLR) aims at identifying, evaluating, and interpreting all available studies to answer particular research questions, and requires more in-depth analysis (Kitchenham and Charters, 2007);
- *Survey* in survey works you should review the published papers and analyze, summarize, organize, and present findings in a **novel** way that can generate an original view to a certain aspect of the domain.

- SCI &EI: 只允许JCR 2区及以上刊物
 - SCI (Science Citation Index) : Web of Science
 - EI (Engineering Index): Engineering Village
 - 中科院分区: Letpub
 - IF (Impact factor) : JCR (Journal Citation Reports)
 - CCF推荐国际学术会议: 只允许CCF-B级及以上会议论文
- CCF中文T级期刊: 只允许T1刊物

- 时间： 只允许2021-2022年,
- 期刊： 优先高被引和热点论文
 - 数据库： IEEE, ACM, Elsevier, Springer...
 - 最新IF
 - SCI分区, 中科院分区
 - CCF等级
- 会议： CCF等级
- 作者： 单位, 通讯作者或导师信息 (院士? 会士? 领域专家?)

Journal Impact Factor

The Journal Impact Factor (JIF) is a journal-level metric calculated from data indexed in the Web of Science Core Collection. It should be used with careful attention to the many factors that influence citation rates, such as the volume of publication and citations characteristics of the subject area and type of journal. The Journal Impact Factor can complement expert opinion and informed peer review. In the case of academic evaluation for tenure, it is inappropriate to use a journal-level metric as a proxy measure for individual researchers, institutions, or articles. [Learn more](#)

2021 JOURNAL IMPACT FACTOR

9.322

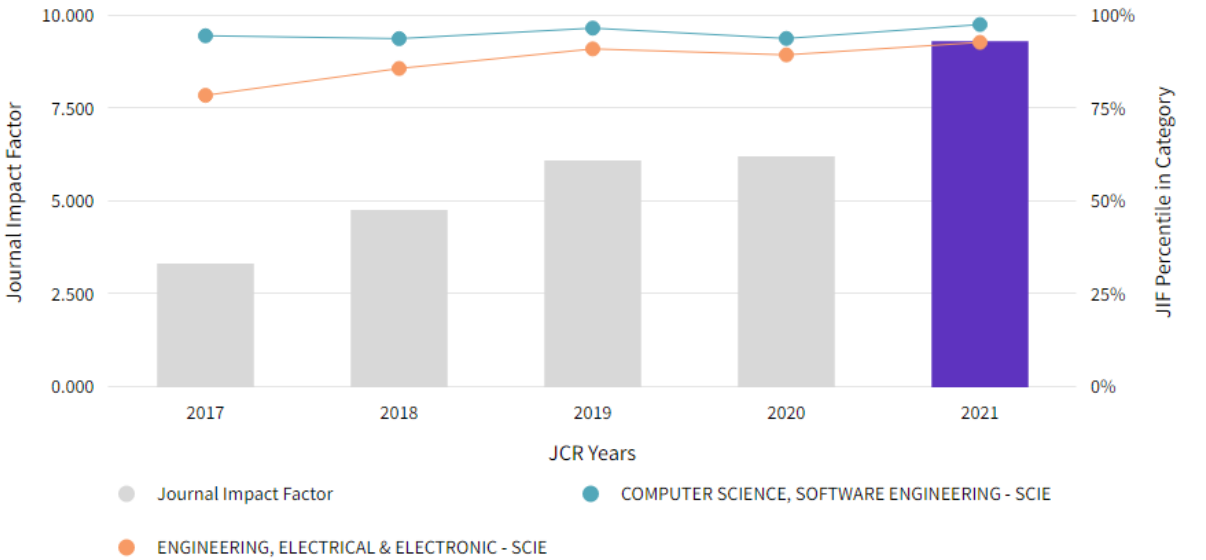
[View calculation](#)

JOURNAL IMPACT FACTOR WITHOUT SELF CITATIONS

8.814

[View calculation](#)

Journal Impact Factor Trend 2021



[View all years](#)

Journal Impact Factor contributing items

[Export](#)

Citable items (118)		Citing Sources (244)	
TITLE		CITATION COUNT	
Title			
The Impact of Automated Parameter Optimization on Defect Prediction Models		51	
Automatic Software Repair: A Survey		41	
Coverage-Based Greybox Fuzzing as Markov Chain		36	
A Comprehensive Investigation of the Role of Imbalanced Learning for Software Defect Prediction		35	
The Impact of Class Rebalancing Techniques on the Performance and Interpretation of Defect Prediction Models		30	
A Systematic Literature Review and Meta-analysis on Cross Project Defect Prediction		30	
Developer Testing in the IDE: Patterns, Beliefs, and Behavior		17	
Deep Semantic Feature Learning for Software Defect Prediction		14	

Rank by Journal Impact Factor

Journals within a category are sorted in descending order by Journal Impact Factor (JIF) resulting in the Category Ranking below. A separate rank is shown for each category in which the journal is listed in JCR. Data for the most recent year is presented at the top of the list, with other years shown in reverse chronological order. [Learn more](#)

EDITION

Science Citation Index Expanded (SCIE)

CATEGORY

COMPUTER SCIENCE, SOFTWARE ENGINEERING

3/110

JCR YEAR	JIF RANK	JIF QUARTILE	JIF PERCENTILE	
2021	3/110	Q1	97.73	
2020	7/108	Q1	93.98	
2019	4/108	Q1	96.76	
2018	7/107	Q1	93.93	
2017	6/104	Q1	94.71	

EDITION

Science Citation Index Expanded (SCIE)

CATEGORY

ENGINEERING, ELECTRICAL & ELECTRONIC

20/276

JCR YEAR	JIF RANK	JIF QUARTILE	JIF PERCENTILE	
2021	20/276	Q1	92.93	
2020	29/273	Q1	89.56	
2019	24/266	Q1	91.17	
2018	38/266	Q1	85.90	
2017	56/260	Q1	78.65	

WOS期刊SCI分区 ⓘ (2021-2022年最新版)	WOS分区等级：1区											
	<table><tr><th colspan="2">按学科分区</th></tr><tr><td>ENGINEERING, ELECTRICAL & ELECTRONIC</td><td>Q1</td></tr><tr><td>COMPUTER SCIENCE, SOFTWARE ENGINEERING</td><td>Q1</td></tr></table>				按学科分区		ENGINEERING, ELECTRICAL & ELECTRONIC	Q1	COMPUTER SCIENCE, SOFTWARE ENGINEERING	Q1		
按学科分区												
ENGINEERING, ELECTRICAL & ELECTRONIC	Q1											
COMPUTER SCIENCE, SOFTWARE ENGINEERING	Q1											
中科院《国际期刊预警名单（试行）》名单 ⓘ	2021年12月发布的2021版：不在预警名单中 2021年01月发布的2020版：不在预警名单中											
中科院SCI期刊分区 (2021年12月最新基础版)	登录查看中科院JCR分区趋势图 <table><tr><th>大类学科</th><th>小类学科</th><th>Top期刊</th><th>综述期刊</th></tr><tr><td>工程技术 2区</td><td>COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 2区</td><td>是</td><td>否</td></tr></table>				大类学科	小类学科	Top期刊	综述期刊	工程技术 2区	COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 2区	是	否
大类学科	小类学科	Top期刊	综述期刊									
工程技术 2区	COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 2区	是	否									
中科院SCI期刊分区 (2021年12月最新升级版) ⓘ	<table><tr><th>大类学科</th><th>小类学科</th><th>Top期刊</th><th>综述期刊</th></tr><tr><td>计算机科学 1区</td><td>COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 1区</td><td>是</td><td>否</td></tr></table>				大类学科	小类学科	Top期刊	综述期刊	计算机科学 1区	COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 1区	是	否
大类学科	小类学科	Top期刊	综述期刊									
计算机科学 1区	COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 1区	是	否									
中科院SCI期刊分区 (2020年12月旧的升级版)	<table><tr><th>大类学科</th><th>小类学科</th><th>Top期刊</th><th>综述期刊</th></tr><tr><td>计算机科学 1区</td><td>COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 1区</td><td>是</td><td>否</td></tr></table>				大类学科	小类学科	Top期刊	综述期刊	计算机科学 1区	COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 1区	是	否
大类学科	小类学科	Top期刊	综述期刊									
计算机科学 1区	COMPUTER SCIENCE, SOFTWARE ENGINEERING 计算机：软件工程 1区 ENGINEERING, ELECTRICAL & ELECTRONIC 工程：电子与电气 1区	是	否									

中国计算机学会推荐国际学术期刊

(软件工程/系统软件/程序设计语言)

一、A 类

序号	刊物简称	刊物全称	出版社	网址
1	TOPLAS	ACM Transactions on Programming Languages & Systems	ACM	http://dblp.uni-trier.de/db/journals/toplas/
2	TOSEM	ACM Transactions on Software Engineering and Methodology	ACM	http://dblp.uni-trier.de/db/journals/tosem/
3	TSE	IEEE Transactions on Software Engineering	IEEE	http://dblp.uni-trier.de/db/journals/tse/

中国计算机学会推荐国际学术会议

(软件工程/系统软件/程序设计语言)

一、A类

序号	会议简称	会议全称	出版社	网址
1	PLDI	ACM SIGPLAN Conference on Programming Language Design & Implementation	ACM	http://dblp.uni-trier.de/db/conf/pldi/
2	POPL	ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages	ACM	http://dblp.uni-trier.de/db/conf/popl/
3	FSE/ESEC	ACM SIGSOFT Symposium on the Foundation of Software Engineering/ European Software Engineering Conference	ACM	http://dblp.uni-trier.de/db/conf/sigsoft/
4	SOSP	ACM Symposium on Operating Systems Principles	ACM	http://dblp.uni-trier.de/db/conf/sosp/
5	OOPSLA	Conference on Object-Oriented Programming Systems, Languages, and Applications	ACM	http://dblp.uni-trier.de/db/conf/oopsla/
6	ASE	International Conference on Automated Software Engineering	IEEE/ACM	http://dblp.uni-trier.de/db/conf/kbse/
7	ICSE	International Conference on Software Engineering	ACM/IEEE	http://dblp.uni-trier.de/db/conf/icse/
8	ISSTA	International Symposium on Software Testing and Analysis	ACM	http://dblp.uni-trier.de/db/conf/issta/
9	OSDI	USENIX Symposium on Operating Systems Design and Implementations	USENIX	http://dblp.uni-trier.de/db/conf/osdi/

• 题目、关键词、摘要

- 解释专业术语
- 解决挑战或问题
- 贡献
- 提出方法或技术
- 实验分析
 - 数据集
 - 结果
 - 结果分析



Contents lists available at ScienceDirect

Journal of King Saud University –
Computer and Information Sciences

journal homepage: www.sciencedirect.com



STORE: Security Threat Oriented Requirements Engineering Methodology

Md Tarique Jamal Ansari ^{a,*}, Dharendra Pandey ^a, Mamdouh Alenezi ^b

^a Department of Information Technology, BBA University, India

^b College of Computer & Information Sciences, Prince Sultan University, Saudi Arabia

ARTICLE INFO

Article history:

Received 29 June 2018

Revised 16 October 2018

Accepted 15 December 2018

Available online 17 December 2018

Keywords:

Security requirements engineering

Threat modeling

Security requirements elicitation

Software security

Security attack analysis

ABSTRACT

As we are continuously depending on information technology applications by adopting electronic channels and software applications for our business, online transaction and communication, software security is increasingly becoming a necessity and more advanced concern. Both the functional and non-functional requirements are important and provide the necessary needs at the early phases of the software development process, specifically in the requirement phase. The aim of this research is to identify security threats early in the software development process to help the requirement engineer elicit appropriate security requirements in a more systematic manner throughout the requirement engineering process to ensure a secure and quality software development. This article proposes the STORE methodology for security requirement elicitation based on security threats analysis, which includes the identification of four points: PoA, PoB, PoC and PoD for effective security attack analysis. Further, the proposed STORE methodology is also validated by a case study of an ERP System. We also compare our STORE methodology with two existing techniques, namely, SQUARE and MOSRE. We have shown that more effective and efficient security requirements can be elicited by the STORE methodology and that it helps the security requirement engineer to elicit security requirements in a more organized manner.

© 2018 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Quotation, Citation & Reference

- Quatation
 - “想象力比知识更重要，”
- Citation & Reference
 - [1] **, **, **. *****, ***, 2020,
()**: **-**.

SC issues arise from code changes of API implementations, which are common in popular libraries. Many of these changes are too subtle for developers to understand their effects on program semantics [2]. Existing test suites may not help effectively expose such differences neither. As such, SC issues are difficult to diagnose. For example, a developer left the following comment in the pull request [10] of the aforementioned issue:

“I have encountered these types of semantic inconsistency issues lots of times when dealing with dependency conflicts. When such issues happen, signature changes can be detected by static analyzers. However, semantic changes would be more difficult to detect. Empirically, developers diagnose them by reading the git history of the library or dynamic testing.”

谢谢!